

The Web & HTML Foundations

How the internet works, and how we write the structure of every web page — from the first request to a complete, semantic document.

Internet & HTTP

DNS

HTML5 Tags

Forms & Tables

Semantic Layout

What we'll build today



1 • Internet & Web

How the internet works, IP addresses, and how DNS turns names into numbers.



2 • Environment

Setting up VS Code, Emmet, and Live Server so your browser updates as you type.



3 • HTML Core

The skeleton: tag anatomy, structure, text, lists, links and images.



4 • Data & Interaction

Tables for data and forms — exactly how users will send data to our backend later.



5 • Modern Standards

Semantic tags, scripting, native interactivity, and a portfolio mini-project.

1 · Internet & Web

2 · Environment

3 · HTML Core

4 · Data & Interaction

5 · Modern Standards

PART 1

The Internet & The Web

Before we write a single tag, let's understand what actually happens when you open a website.



The Restaurant Analogy

The internet is a network of computers talking to each other. Your browser (the **client**) asks another computer (the **server**) for a page; the server sends it back. Data travels in small **packets** that hop through **routers** until they arrive.



A request, then a response

Every page load is a conversation in **HTTP** — the client sends a **request**; the server returns a **response**.

REQUEST

GET /index.html HTTP/1.1

Host: infinitibit.com

Accept: text/html

User-Agent: Chrome/126

RESPONSE

200 OK HTTP/1.1

Content-Type: text/html

Content-Length: 1024

...

GET — fetch a page

POST — send data

200 OK · 404 Not Found · 500 Error

IP — the address of a machine

An **IP address** (Internet Protocol) is the unique mailing address of a device on a network — it's how data knows where to go.

- ◆ **IPv4** — four numbers, e.g. `142.250.190.78` (~4.3 billion addresses)
- ◆ **IPv6** — much longer, e.g. `2400:cb00:...` (effectively unlimited)
- ◆ **localhost** = `127.0.0.1` — your own machine. You'll use this constantly in MERN, e.g. `localhost:3000`

```
addresses

<!-- Your React app in development -->
http://localhost:3000

<!-- Your Express API -->
http://localhost:5000/api/users

<!-- A real server on the internet -->
https://142.250.190.78
```



Analogy: an IP address is like a home mailing address or a GPS coordinate — it pinpoints exactly one location.

DNS — the internet's phonebook

The **Domain Name System (DNS)** translates human-friendly names like **google.com** into the machine IP addresses computers actually use to connect.

💡 **Analogy:** DNS is your phone's contact list. You don't memorise a friend's 10-digit number (the IP) — you just tap their name (the domain), and the phone dials the number for you.

google.com

Human-readable name



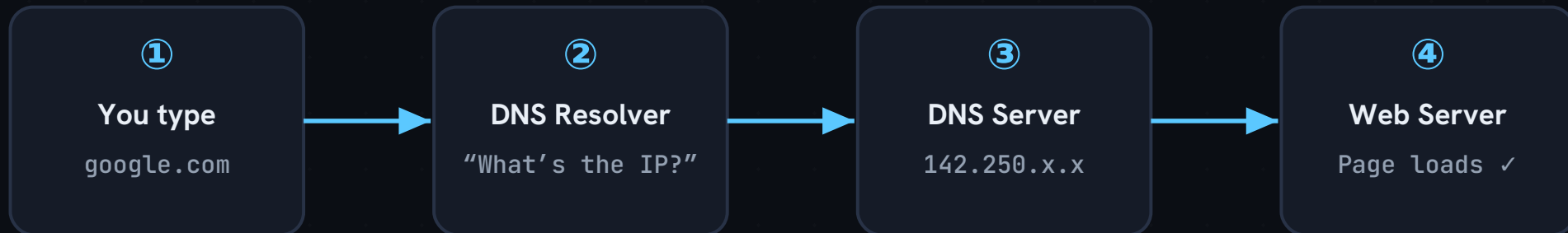
142.250.190.78

Machine IP address

From name to number, in milliseconds

When you type a domain, your browser quietly runs a lookup **before** the page can load:

DNS = the phonebook of the internet: name → number



Domain name (easy to remember) → IP address (how machines connect)

1 · Internet & Web

2 · Environment

3 · HTML Core

4 · Data & Interaction

5 · Modern Standards

PART 2

The Environment

Let's get your machine ready. From here on, follow along and type everything yourself.

Your workshop, ready in 60 seconds

- ◆ Create a folder for your project and open it in VS Code
- ◆ Create a file named `index.html` — the default home page
- ◆ Type `!` then **Tab** — Emmet generates the full boilerplate instantly
- ◆ Install the **Live Server** extension → right-click → Open with Live Server

✚ **The "aha!" moment:** with Live Server running, hit **Save** and watch the browser refresh on its own. Instant feedback makes learning fun.

```
index.html

<!-- type "!" then press Tab -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

1 · Internet & Web

2 · Environment

3 · HTML Core

4 · Data & Interaction

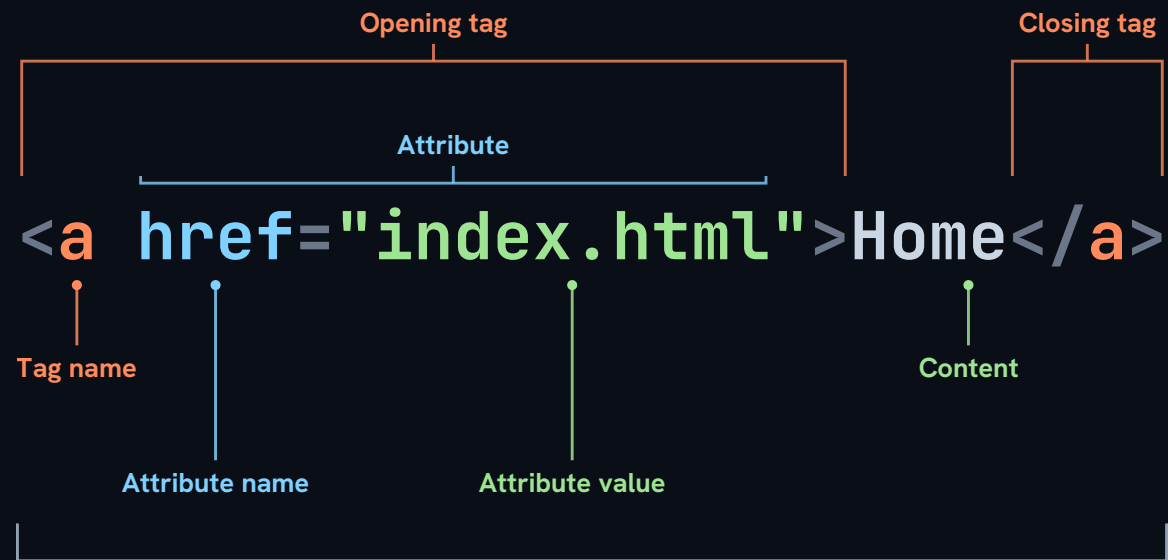
5 · Modern Standards

PART 3

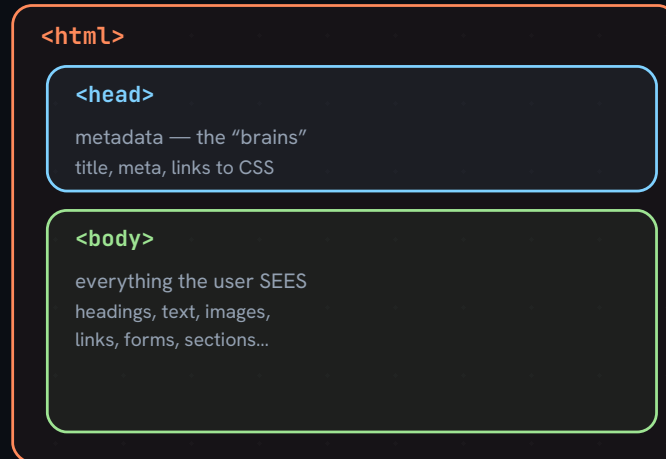
HTML Core — The Skeleton

HTML gives a page its structure. We'll learn the building blocks one tag at a time.

Anatomy of an HTML tag



The four bones of every page



- ◆ **<html>** — the root that wraps the whole page
- ◆ **<head>** — metadata & setup the user never sees (the "brains")
- ◆ **<title>** — the text shown on the browser tab
- ◆ **<body>** — everything the user actually sees on screen

Remember: **head** is for the browser; **body** is for the human.

Headings & paragraphs

- ◆ `<h1>` ... `<h6>` — headings, large to small (one `<h1>` per page)
- ◆ `<p>` — a paragraph of text
- ◆ `<strong>` — important text (bold)
- ◆ `<em>` — emphasised text (italic)

▶ **Your turn (2 min):** add an `<h1>` with your name and a short `<p>` about yourself.

text.html

```
<h1>Saddat Hasan</h1>
<h2>Full-Stack Engineer</h2>

<p>I build web apps with the
  <strong>MERN stack</strong> and
  <em>love teaching</em> beginners.</p>
```

Ordered & unordered lists

- ◆ `<ul>` — unordered list (bullets) for items with no sequence
- ◆ `<ol>` — ordered list (numbers) for steps or rankings
- ◆ `<li>` — a single list item, nested inside either one

Both list types hold `<li>` items — only the **marker** changes. Lists can also be nested inside one another.

```
lists.html

<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<ol>
  <li>Learn the frontend</li>
  <li>Learn the backend</li>
  <li>Connect them</li>
</ol>
```

• LINKS

The anchor tag & href

The anchor tag `<a>` creates a link. Its **href** attribute says where it goes.

- ◆ **Absolute URL** — a full address to another site:

`https://infinitebit.com`

- ◆ **Relative URL** — a path inside your own project:

`about.html`

```
links.html

<!-- Absolute: leaves your site -->
<a href="https://google.com">
  Search Google
</a>

<!-- Relative: another page of yours -->
<a href="about.html">
  About Me
</a>
```

Images, video & audio

The `<img>` tag embeds an image. It is **self-closing** — no closing tag needed.

- ◆ `src` — the path to the image file
- ◆ `alt` — text shown if the image fails & read aloud by screen readers
- ◆ Also: `<video>` and `<audio>` for multimedia

💡 **Always write alt text** — it's vital for accessibility and when an image can't load.

```
media.html



<video src="demo.mp4" controls></video>

<audio src="intro.mp3" controls></audio>
```

1 · Internet & Web

2 · Environment

3 · HTML Core

4 · Data & Interaction

5 · Modern Standards

PART 4

Data & Interaction

Tables and forms are more complex — they nest several specific tags. Let's slow down.

• TABLE

Tables — rows, headers & cells

- ◆ `<table>` — wraps the whole table
- ◆ `<tr>` — a table row
- ◆ `<th>` — a header cell (bold)
- ◆ `<td>` — a normal data cell

💡 **Trap to avoid:** tables are for **tabular data** only — never for page layout. (That's CSS's job, next lecture.)

```
table.html

<table>
  <tr>
    <th>Day</th>
    <th>Topic</th>
  </tr>
  <tr>
    <td>Mon</td>
    <td>HTML</td>
  </tr>
</table>
```

Forms — how users send data

- ◆ `<form>` — groups inputs that submit together
- ◆ `<label>` — a caption tied to an input
- ◆ `<input>` — a field; `type` sets its kind
- ◆ `<button>` — submits the form

`type="text"`

`"email"`

`"password"`

`"submit"`



Look ahead: when we reach Node.js & Express, this is **exactly** how users will send data to our database.

```
form.html

<form>
  <label>Email</label>
  <input type="email" name="email">

  <label>Password</label>
  <input type="password" name="pw">

  <button type="submit">Log in</button>
</form>
```

1 · Internet & Web

2 · Environment

3 · HTML Core

4 · Data & Interaction

5 · Modern Standards

PART 5

Modern Web Standards

The difference between an amateur and a professional: writing HTML that has meaning.

HTML with meaning

`<header>`

`<nav>`

`<main>`

`<section>`

`<article>`

`<aside>`

`<footer>`

A `<div>` works, but it's meaningless. **Semantic tags** describe their purpose to browsers, search engines & screen readers.

- ◆ `<header>` `<nav>` — top & navigation
- ◆ `<main>` `<section>` `<article>` — content
- ◆ `<footer>` — bottom of the page

The script tag & where it goes

The `<script>` tag adds JavaScript — the behaviour that makes pages interactive.

- ◆ Place it **just before** `</body>`
- ◆ **Why?** So all the HTML loads first — your script can then find the elements it needs

`</>` **Preview:** JavaScript is a whole lecture of its own — today we just learn where it lives.

```
index.html

<body>
  <h1>Hello</h1>

  <!-- everything above is loaded,
        now run the script -->
  <script src="app.js"></script>
</body>
```

Interactivity without JavaScript

Some interactivity needs **no JavaScript at all** — modern HTML has it built in.

- ◆ `<details>` + `<summary>` — a click-to-expand accordion
- ◆ `<dialog>` — a native pop-up modal

```
interactive.html

<details>
  <summary>What is the MERN stack?</summary>
  <p>MongoDB, Express, React, Node.</p>
</details>

<dialog open>
  <p>Welcome to the course!</p>
</dialog>
```

• SPECIAL PURPOSE TAGS

The invisible essentials

- ◆ `<meta charset="UTF-8">` — supports every character & emoji
- ◆ `<meta name="viewport">` — makes the page responsive on phones
- ◆ `<link>` — connects external files, most often your CSS

These live in the `<head>` — invisible, but essential for a page to work correctly everywhere.

```
head.html

<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width,
                initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
</head>
```



Build a Personal Portfolio

Combine everything into a structured, semantic **Personal Portfolio** page — pure HTML, no CSS yet.



Structure it

Use `header`, `nav`, `main`, `footer`.



Fill it

A heading, an intro paragraph, a list of skills, and a profile image.



Connect it

Add links and a small contact `form`.



Celebrate the ugly! Raw HTML is supposed to look plain. Next lecture, CSS makes it beautiful.

● RECAP

Key takeaways

- ◆ The web is a **client** asking a **server** over **HTTP**
- ◆ **DNS** turns a domain name into an **IP address**
- ◆ Every tag has a clear **anatomy**: open, attributes, content, close
- ◆ `<head>` is for the browser, `<body>` for the human
- ◆ Text, lists, links & images are your everyday building blocks
- ◆ **Tables** are for data; **forms** send data to a server
- ◆ **Semantic tags** give your HTML meaning
- ◆ Next stop: **CSS** — making it beautiful

Practice makes it stick



I do

I demonstrate a tag and explain why it exists before how to type it.



We do

We write it together, line by line, on screen.



You do

You get a 2-minute challenge to make it your own.

Homework: finish your **Personal Portfolio** page using only the tags from today. Bring questions — next lecture we style it with **CSS**.

• END OF LECTURE 01

Thank you. Questions?

You now understand how the web works and how to structure any page in HTML. Next lecture —
CSS: making it beautiful.



Saddat Hasan

Full-Stack Engineer · InfinitiBit GmbH